

AI WEATHERWISE

Description

The AI WeatherWise is a robust RESTful backend platform built with **Node.js** and **Express.js**, utilizing **MongoDB** and **Mongoose ODM** for data persistence. It delivers real-time weather metrics integrated with **Google Gemini AI** to provide intelligent summaries and personalized recommendations based on live environmental conditions.

The platform features secure JWT authentication, bcrypt password hashing, and a comprehensive suite of endpoints for managing favorite locations and fetching weather insights. It includes a resilient fallback mode ensuring API functionality even without external API keys, alongside centralized error handling and request sanitization.

Main Purpose of the Project

AI Weather-wise allows users to manage the city you are live in , fetch real-time weather. The system converts raw weather metrics into easier-to-understand summaries and recommendations.

Problem

- Difficulty tracking and managing multiple favorite weather locations efficiently.
- Lack of personalized recommendations for outdoor activities or clothing.
- Need for natural language weather summaries rather than just raw data metrics.
- Absence of a secure system to store and sync preferences across devices.

Solution

The AI WeatherWise provides a centralized service that handles weather data retrieval and AI orchestrations. It manages a secure User and Location database, allowing for customized insights and automated recommendation workflows through Google Gemini.

Team ID : SWTID-2026-7338

NAME	Email-id	Nm-id
Rithick Kumar S (Leader)	rithickkumar308@ gmail.com	5C5D638CCEC7D63BD6448AB9295A0E7A
Nithish Kumar K (Member)	nithishkumar6380@ gmail.com	0C13338CF1F87018EC74DEFDF36F707B
Praveen V (Member)	praveenvenkatesan13@ gmail.com	0294EBC6A448EE8B7424B5E7F474A08C
Ruthreshwaran S (Member)	ruthreshwaran825@ gmail.com	F456B8241D0A6419233429A64222B49

PHASE – 1

Software Requirements

Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Gemini AI SDK:** Integration using @google/generative-ai for natural language insights.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

Technologies Used

- Visual Studio Code – write and Runs the Backend code.
- Node.js – Runs the backend server and application logic.
- Express.js – Handles the REST APIs, routes, requests, and responses.
- MongoDB – Stores users and favorite location data.
- Mongoose – Connects the Node.js application with MongoDB and manages schemas.
- OpenWeatherMap – Provides real-time weather data.
- Google Gemini AI – Generates weather summaries and personalized recommendations.
- JWT – Provides user authentication and protects secured routes.
- bcrypt – Hashes user passwords securely.
- Postman – Used to test the REST APIs.

Short Explanation

“AI WeatherWise is a Node.js and Express.js based RESTful backend application that uses MongoDB for data storage, OpenWeatherMap for real-time weather data, and Google Gemini AI to generate intelligent weather summaries and personalized recommendations.”

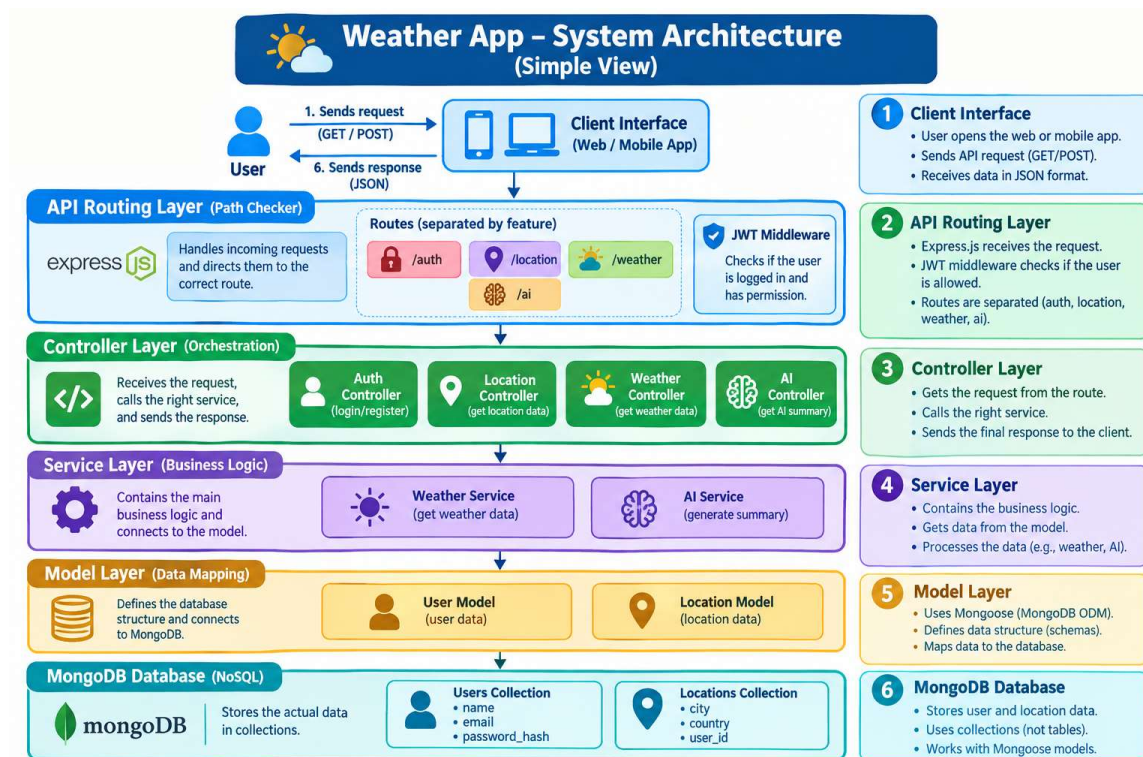
PHASE – 2

Hardware Requirements

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

PROJECT ARCHITECTURE



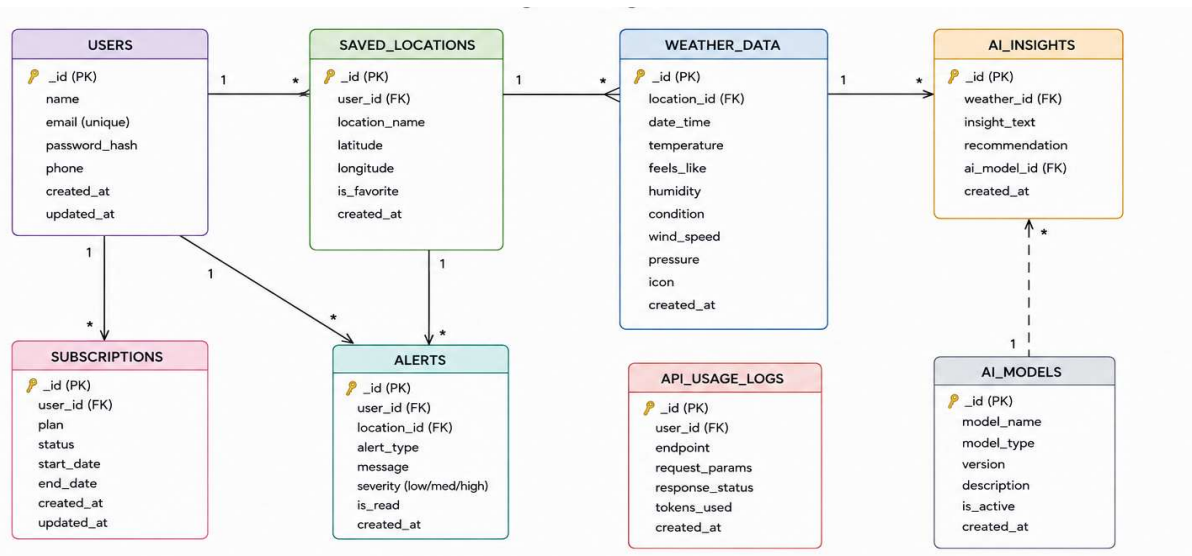
The AI WeatherWise application follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenWeatherMap and generation of AI insights via Gemini, while the Model layer manages User and Location entities.

Components Definition

- **Interface:** API tools like Postman query distinct route paths to extract JSON entities.

- **Express Server Gateway:** Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.
- **Authentication & Role Matrix Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds for guarded resource paths.
- **Granular Services (Logic Modules):** Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators.
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries.
- **AI Service Integration:** Leverages @google/generative-ai to generate weather summaries and personalized recommendations based on live temperature and conditions.

Entity-Relationship (ER) Diagram Description



The data architecture manages User profiles and their associated favorite Locations through logical references:

1. User Entity

- _id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Location Entity

- `_id` (Primary Key - ObjectId)
- `user` (Foreign Key referencing User Collection)
- `city` (String, Required)
- `country` (String, Required)
- `createdAt` (Date, Default: Now)

PHASE – 3

Project Setup and Configuration

Creating the Project Folder

- Create a new folder named **AI Weatherwise**
- Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies.

Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

Install Nodemon for development:

```
npm install --save-dev nodemon
```

Environment Configuration

Create a **.env** file in the project root and configure the required environment variables.

```
PORT=5000  
MONGO_URI=your_mongodb_connection_string  
JWT_SECRET=your_jwt_secret  
GEMINI_API_KEY=your_gemini_api_key
```

PHASE – 4

Database Configuration

API Testing Demo :

1. Configure MongoDB:

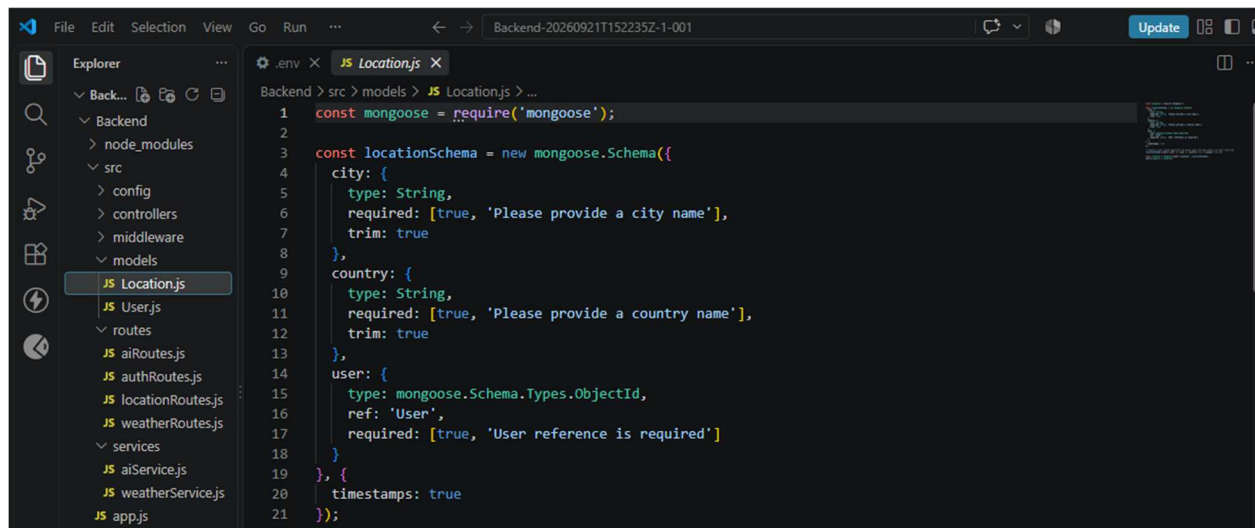
- Install Mongoose.
In your Node.js project directory, run:
`npm install mongoose`
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

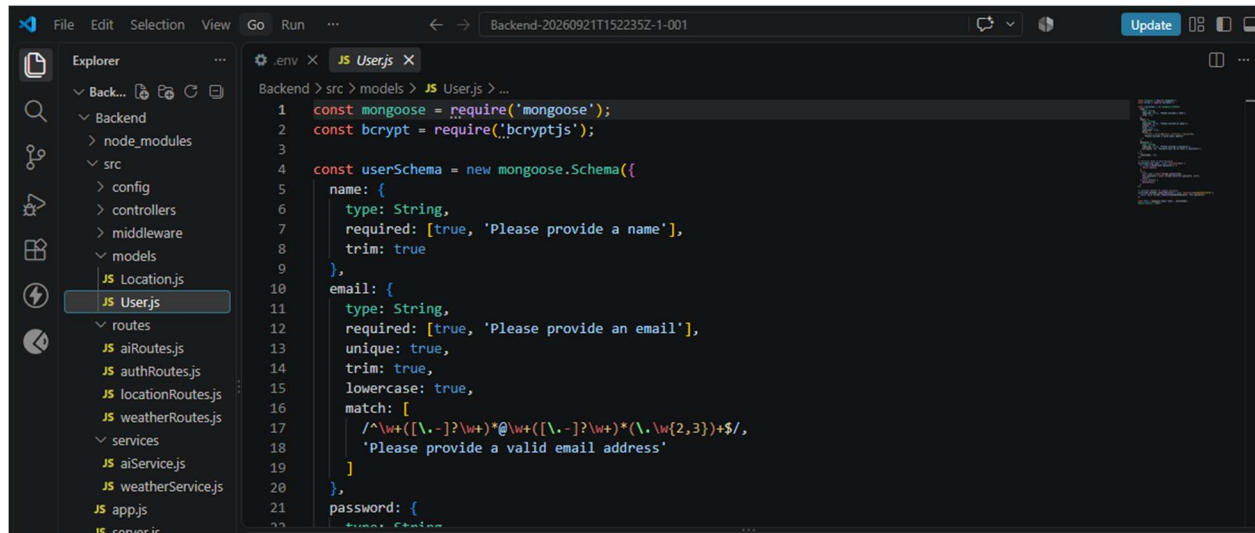
Mongoose Schema Definitions

Location Schema:

A screenshot of a Visual Studio Code editor window. The Explorer sidebar on the left shows a project structure with folders 'Backend' and 'src', and subfolders 'node_modules', 'config', 'controllers', 'middleware', 'models', 'routes', and 'services'. The 'models' folder is expanded, showing 'Location.js' selected. The main editor area displays the content of 'Location.js'. The code defines a Mongoose schema for a location, with fields for 'city' and 'country' (both strings, required, and trimmed) and a 'user' field (a reference to the 'User' model, required). It also includes timestamps. The code is as follows:

```
1 const mongoose = require('mongoose');
2
3 const locationSchema = new mongoose.Schema({
4   city: {
5     type: String,
6     required: [true, 'Please provide a city name'],
7     trim: true
8   },
9   country: {
10    type: String,
11    required: [true, 'Please provide a country name'],
12    trim: true
13  },
14   user: {
15     type: mongoose.Schema.Types.ObjectId,
16     ref: 'User',
17     required: [true, 'User reference is required']
18   }
19 }, {
20   timestamps: true
21 });
```

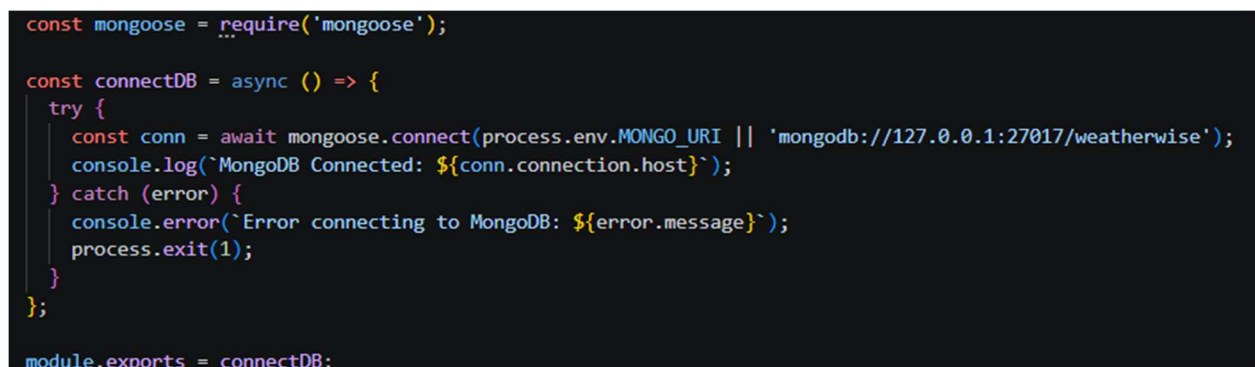
User schema

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project structure with folders 'Backend' and 'src'. Under 'src', there are folders 'config', 'controllers', 'middleware', 'models', 'routes', and 'services'. The 'models' folder is expanded, showing 'Location.js' and 'User.js'. 'User.js' is selected and its content is displayed in the main editor. The code defines a Mongoose schema for a user with fields for name, email, and password. The name field is a string, required, and trim. The email field is a string, required, unique, trim, and lowercase, with a regex validation for a valid email address. The password field is a string, required, and has a minimum length of 6 characters.

```
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcryptjs');
3
4 const userSchema = new mongoose.Schema({
5   name: {
6     type: String,
7     required: [true, 'Please provide a name'],
8     trim: true
9   },
10  email: {
11    type: String,
12    required: [true, 'Please provide an email'],
13    unique: true,
14    trim: true,
15    lowercase: true,
16    match: [
17      /^\/\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
18      'Please provide a valid email address'
19    ]
20  },
21  password: {
22    type: String,
```

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

A screenshot of the Visual Studio Code editor showing a JavaScript file for database connection. The code uses Mongoose to connect to a MongoDB instance. It defines an asynchronous function 'connectDB' that attempts to connect using 'process.env.MONGO_URI' or a default URI. If the connection is successful, it logs the connection details. If there is an error, it logs the error message and exits the process. The function is then assigned to 'module.exports'.

PHASE - 5

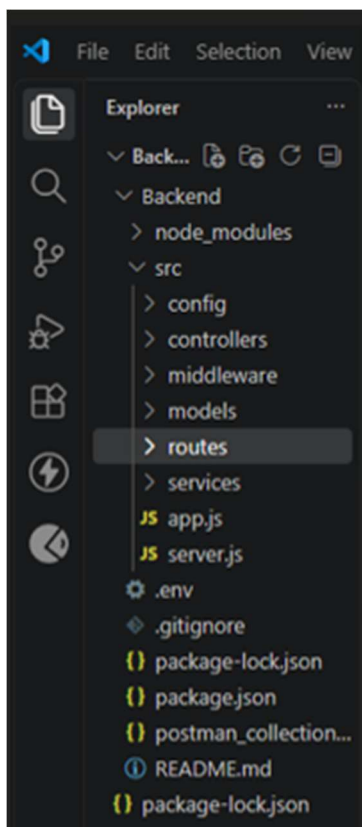
Project Executions

Project Execution is AI weatherwise can be executed by using Node.js before doing the we must make sure that MongoDB is running. The REST APIs can be tested by using Postman or Thunder Client.

Before running the Backend server:

- By install all requirement by the command
`npm install`
- Start the development server by the command
`npm start`

A overview of the Backend



PHASE – 6
API testing

Register API:

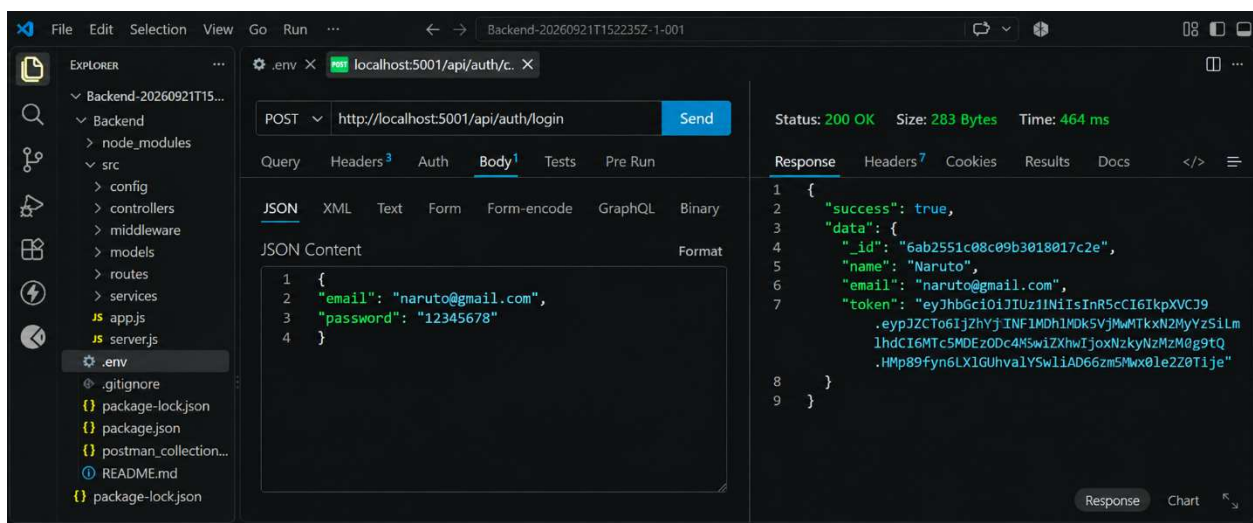
Route: GET/api/auth/register

```
{  
  "name": "Naruto",  
  "email": "naruto@gmail.com",  
  "password": "12345678"  
}
```

Login API:

Route: POST/api/auth/login

```
{  
  "email": "naruto@gmail.com",  
  "password": "12345678"  
}
```



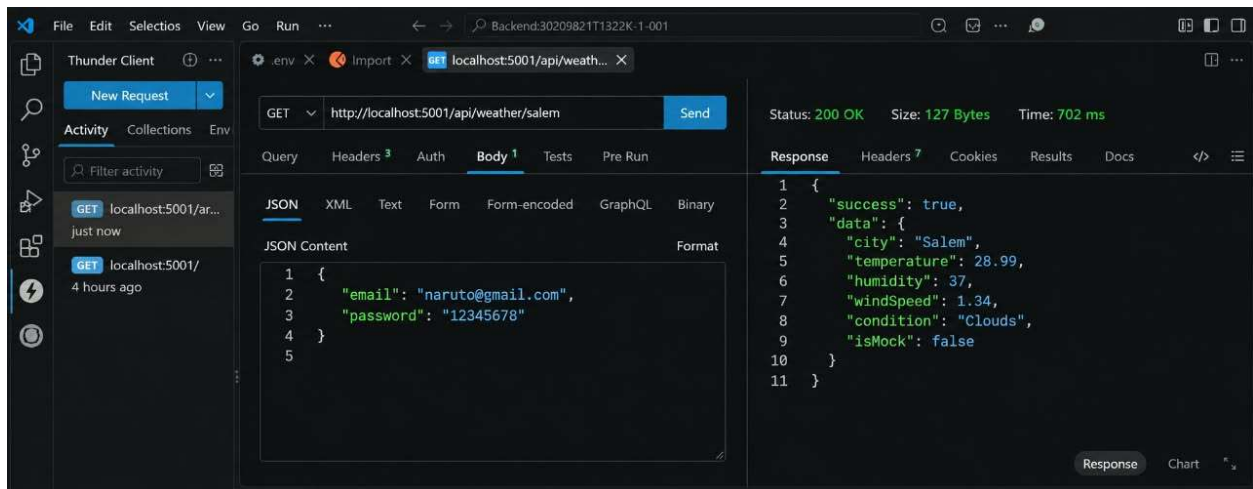
PHASE – 7

**AI Intelligent content
(for specification demo)**

Weather API for city:

Route: GET /api/weather/salem

```
{  
  "email": "naruto@gmail.com",  
  "password": "12345678"  
}
```



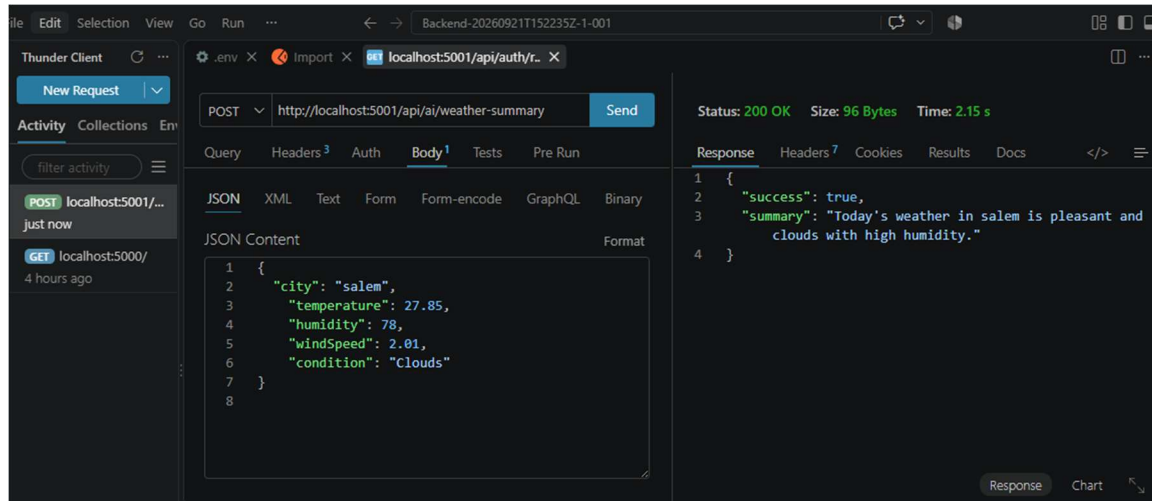
Here the real time weather condition of Salem. A client can request a city's weather using an API endpoint such as GET /api/weather/Salem. The Express.js backend receives the request, obtains the required weather information, processes it, and returns a JSON response to the client.

Weather summary API:

Route: POST /api/ai/weather-summary

```
{  
  "city": "salem",  
  "temperature": 27.85,
```

```
    "humidity": 78,  
    "windSpeed": 2.01,  
    "condition": "Clouds"  
}
```



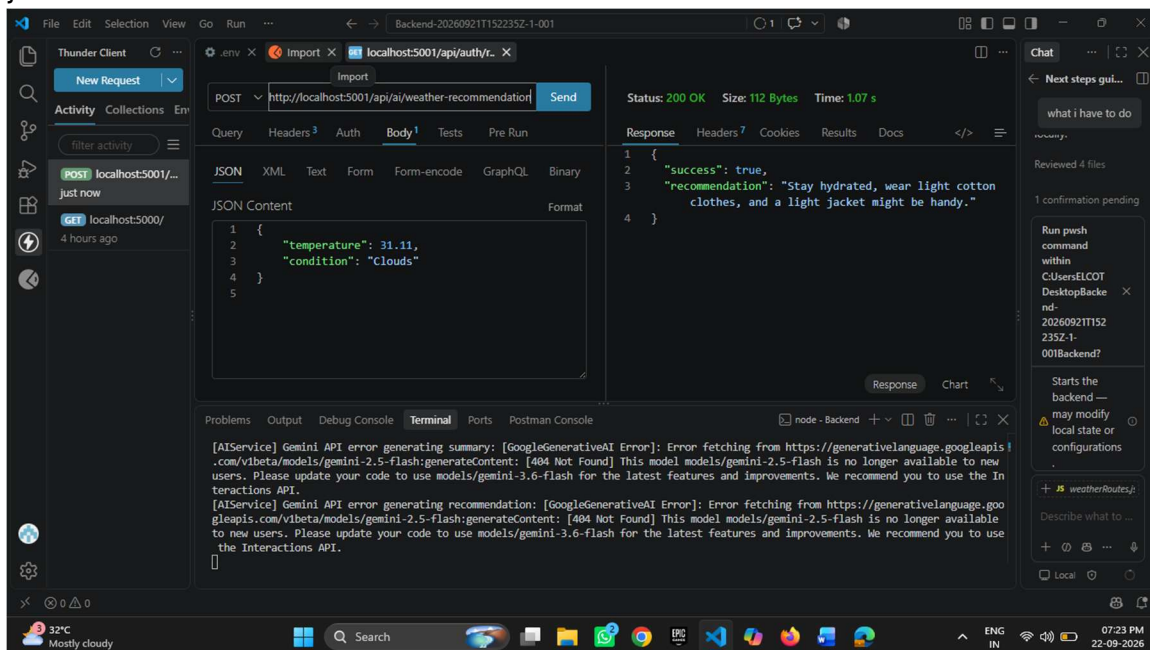
Here the weather of city: Salem, 28.93 Cloudy. The backend can send these weather details to Gemini AI. Gemini can generate a natural-language summary and a context-aware recommendation, such as suggesting hydration or suitable planning for outdoor activities.

PHASE – 8
Last API test
(Weather-recommendation)

Weather-recommendation API

Route: POST /api/ai/weather-recommendation

```
{  
  "temperature": 31.11,  
  "condition": "Clouds"  
}
```



This project is primarily a backend REST API project. API responses can be viewed as JSON in a browser when an API endpoint is opened, and APIs can be tested using Postman or Thunder Client. The provided project document does not describe a complete frontend dashboard or specific weather widgets.

Demo Video Link: https://drive.google.com/file/d/1P_mnQIf-pP6UtvVWPt908xVrZgFbZdfm/view?usp=drive_link